

Internship Project Report

LendSwift — Multi-Step Loan Application Form

Intern: Cleven Samwel

Role: Frontend Developer

Project Type: Full-stack Front-End Development

Tech Stack: Vite + React 19 + Tailwind CSS 3 + react-hook-form + Zod + Cypress

Internship Project Report

LendSwift — Multi-Step Loan Application Form

Intern: Cleven Samwel **Role:** Frontend Developer **Duration:** 15/06/2026 – 30/06/2026 **Project Type:** Full-stack Front-End Development **Tech Stack:** Vite + React 19 + Tailwind CSS 3 + react-hook-form + Zod + Cypress

Table of Contents

1. [Executive Summary](#)
 2. [Project Overview](#)
 3. [Work Completed](#)
 4. [Architecture & Design](#)
 5. [Key Features Implemented](#)
 6. [Testing & Quality Assurance](#)
 7. [Security & Compliance](#)
 8. [Skills & Technologies Gained](#)
 9. [Challenges & Solutions](#)
 10. [Project Statistics](#)
 11. [Conclusion](#)
-

1. Executive Summary

This report documents the development of **LendSwift**, an 8-step multi-step loan application form wizard designed for a financial institution. The application guides applicants through the complete loan lifecycle — from selecting a loan type and providing personal details, through KYC verification and document upload, to review and digital submission.

The project was built entirely on the front end using modern React patterns, with comprehensive validation, encrypted data persistence, accessibility compliance, and a full suite of 15 Cypress E2E test journeys. All 8 steps are fully implemented with dynamic cross-step validation, conditional logic, and a responsive, accessible UI.

2. Project Overview

2.1 Objectives

- Build a production-ready multi-step loan application form
- Implement conditional form logic based on loan type, employment type, and other factors
- Ensure data security through encryption and PII masking
- Deliver comprehensive E2E test coverage
- Meet WCAG 2.1 AA accessibility standards

2.2 Loan Types Supported

Loan Type	Amount Range	Tenure Range	Interest Rate
Personal	₹50,000 – ₹10,00,000	12–60 months	10.5% p.a.
Home	₹50,000 – ₹1,00,00,000	60–360 months	8.5% p.a.
Business	₹50,000 – ₹50,00,000	12–120 months	14.0% p.a.

2.3 Application Steps

Step	Name	Key Fields
1	Loan Type & Basic Information	Loan type, amount, tenure, purpose, referral code
2	Personal Information	Name, DOB, gender, marital status, email, mobile
3	Identity Verification (KYC)	PAN, Aadhaar (Verhoeff checksum), voter ID, passport
4	Address Information	Current/permanent address, PIN auto-fill, residence type
5	Employment & Income Details	Salaried/Self-Employed/Business Owner sub-forms, GST
6	Co-Applicant & Guarantor	Conditional visibility, PAN verification, signature
7	Document Upload & E-Signature	10 document types, image compression, canvas signature
8	Review, Consent & Pre-Approval	Section review, EMI calculator, 4–5 consents, submit

3. Work Completed

3.1 Source Code — 36 Files in `src/`

Entry & Application Shell (4 files)

File	Purpose
<code>src/main.jsx</code>	React 19 entry point with StrictMode
<code>src/index.css</code>	Tailwind CSS directives and base styles
<code>src/App.jsx</code>	Root component: form state management, auto-save orchestration, resume modal, toast notifications, smart field clearing
<code>src/constants.js</code>	Step registry, loan type definitions, amount/tenure ranges, interest rates, threshold values, field option lists

Wizard & Navigation (3 files)

File	Purpose
src/components/Wizard.jsx	Step registry, navigation guards, URL sync (?step=N), async validation, success modal, Cypress bridge
src/components/ProgressBar.jsx	Accessible step indicator with aria-current , connecting lines
src/components/StepNavigation.jsx	Previous/Next/Submit buttons, disabled states, tooltips

Step Components (8 files)

Each step component handles its own rendering, conditional fields, and integrates with react-hook-form via useStepForm . Validation schemas are dynamically resolved through the schema factory.

Component	File Size	Distinctive Logic
Step1LoanType.jsx	4.8 KB	Dynamic amount ranges, tenure dropdowns, purpose dropdowns per loan type
Step2PersonalInfo.jsx	5.1 KB	Age calculation (21–65), email/mobile verification simulation, alternate mobile must differ
Step3KYC.jsx	3.9 KB	PAN entity type validation, Aadhaar Verhoeff checksum, passport conditional (Home > ₹50L)
Step4Address.jsx	10.0 KB	Dual address sections, PIN auto-fill (112 entries), rent/previous/permanent conditionals
Step5Employment.jsx	7.1 KB	Discriminated union sub-forms, GST validation, Business loan filters Salaried option
Step6CoApplicant.jsx	4.3 KB	Dynamic visibility, Spouse default when married, PAN verification
Step7Documents.jsx	3.1 KB	10 document types with conditional requirements, image compression
Step8Review.jsx	12.4 KB	Section-by-section review, EMI summary card, consent checkboxes, success modal

Common Components (9 files)

All built using the Compound Components pattern with React . forwardRef for RHF compatibility:

Component	Purpose	Accessibility Features
<code>Input.jsx</code>	Text input with Label/Field/Error/HelpText sub-components	<code>aria-invalid</code> , <code>aria-describedby</code> , <code>role="alert"</code>
<code>Select.jsx</code>	Native select with custom chevron	Same ARIA pattern
<code>RadioGroup.jsx</code>	Radio group via <code>fieldset</code> / <code>legend</code>	Horizontal + vertical layouts
<code>Checkbox.jsx</code>	Single checkbox with linked label	Error display
<code>CurrencyInput.jsx</code>	INR input with Indian formatting	<code>inputMode="numeric"</code>
<code>MaskedInput.jsx</code>	PAN/Aadhaar with masking	<code>autoComplete="off"</code> , <code>font-mono</code>
<code>ErrorMessage.jsx</code>	Error display	<code>aria-live="polite"</code> , <code>role="alert"</code>
<code>FileUpload.jsx</code>	Drag-and-drop file upload	Status in <code>aria-live</code> region
<code>SignatureCanvas.jsx</code>	E-signature capture	Blur overlay anti-screen-capture

Custom Hooks (5 files)

Hook	Purpose	Key Behavior
<code>useAutoSave.js</code>	Encrypted auto-save every 30s	AES-256-GCM encryption, debounced, <code>saveInProgress</code> guard
<code>useFormPersistence.js</code>	Resume/Start Fresh on load	72h expiry, Zod validation on resume
<code>useStepForm.js</code>	RHF wrapper with dynamic schema	Resolves schema via <code>schemaFactory</code> per step
<code>useVerification.js</code>	Simulated async verification	1.5s delay, format check, timer cleanup
<code>usePinCodeLookup.js</code>	PIN → city/state/post office	400ms debounce, 112-entry lookup

Validation Schemas (9 files)

Schema	Key Rules
<code>schemaFactory.js</code>	Routes step 0–7 to schema with full form state
<code>step1Schema.js</code>	Dynamic amount/tenure per loan type
<code>step2Schema.js</code>	Age 21–65, mobile regex <code>^[6-9]\d{9}\$</code> , cross-step DOB+tenure ≤ 65
<code>step3Schema.js</code>	PAN format + entity type, Aadhaar Verhoeff, conditional passport
<code>step4Schema.js</code>	Address 5–200 chars, PIN 6 digits, rent/previous/permanent conditionals
<code>step5Schema.js</code>	<code>z.discriminatedUnion</code> , salary $\geq ₹15K$, turnover $\geq ₹3L$, GST 15-char
<code>step6Schema.js</code>	Conditional: empty when hidden, full validation when visible
<code>step7Schema.js</code>	10 document specs, PAN-verified $\rightarrow$ PAN optional
<code>step8Schema.js</code>	4 consents as <code>z.literal(true)</code> , 5th conditional on EMI ratio

Utilities (6 files)

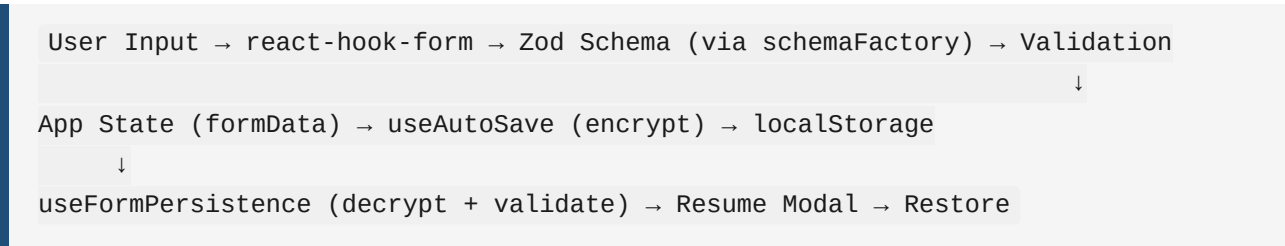
File	Purpose
<code>validators.js</code>	PAN (format + entity type), Aadhaar (Verhoeff checksum with d/p/inv tables), GST, mobile, email
<code>emiCalculator.js</code>	<code>calculateEMI()</code> , <code>getPreApprovalSummary()</code> with processing fee
<code>encryption.js</code>	AES-256-GCM via Web Crypto API
<code>formatCurrency.js</code>	Indian number formatting (Intl.NumberFormat + manual regex)
<code>imageCompressor.js</code>	Canvas API compression pipeline (1200px, quality 0.7 $\rightarrow$ 0.3)
<code>pinCodeData.json</code>	112 PIN entries covering all 28 states + 8 UTs

4. Architecture & Design

4.1 Three-Layer Cross-Step Validation

Layer	Responsibility	Implementation
Schema Layer	Validation-time rules	<code>schemaFactory.js</code> passes <code>formState</code> to each step's Zod schema
State Layer	Change-time data integrity	<code>App.jsx</code> smart-clears dependent fields on loan/employment type change
UI Layer	Render-time visibility	<code>Wizard.jsx</code> computes <code>showCoApplicant</code> , <code>canSubmit</code> reactively

4.2 Data Flow



4.3 Step Registry Pattern

```
STEPS = [
  { id: 0, label: 'Loan Type', icon: ... },
  { id: 1, label: 'Personal Info', icon: ... },
  ...
]

STEP_COMPONENTS = { 0: Step1LoanType, 1: Step2PersonalInfo, ... }
```

Steps are rendered by keying into `STEP_COMPONENTS[currentStep]`. Visibility is computed via `useMemo`, and navigation guards run `await trigger()` before advancing.

4.4 Encryption Flow



5. Key Features Implemented

5.1 Conditional Form Logic

- **Loan type affects:** amount ranges, tenure ranges, purpose dropdowns, employment restrictions, co-applicant visibility, document requirements
- **Employment type affects:** sub-form fields, document requirements, salary/turnover validation
- **Residence type affects:** rent amount field, previous address requirements
- **Marital status affects:** co-applicant default relationship
- **Amount thresholds:** Personal > ₹5L or Business > ₹20L triggers co-applicant step

5.2 Simulated Verification

The `useVerification` hook provides realistic verification UX: - On blur with valid format → 1.5s spinner → green "Verified" badge - On invalid format → immediate red error - Used for: email, mobile, PAN, Aadhaar

5.3 PIN Code Auto-Fill

- 112 PINs across India's 28 states + 8 UTs
- 400ms debounced lookup
- Auto-fills city, state, post office
- User can override auto-filled values

5.4 Client-Side Image Compression

- Canvas API pipeline
- Max 1200px width, quality 0.7, recursive 0.1 steps until < 2MB
- Shows original vs. compressed file sizes
- Non-blocking async execution

5.5 EMI Calculator

```
r = annualRate / 12 / 100
EMI = P × r × (1+r)^n / ((1+r)^n - 1)
Processing Fee = max(2000, min(25000, P × 0.01))
```

Rates: Personal 10.5%, Home 8.5%, Business 14%

5.6 Auto-Save with Encryption

- 30-second debounced auto-save
- AES-256-GCM encryption via Web Crypto API
- localStorage key: `lendswift_draft_[loanType]`
- Draft expiry: 72 hours
- "Draft saved at [time]" toast notification

5.7 Resume/Start Fresh

- On page load, checks for valid draft
- Shows modal: "You have an incomplete application. Resume or Start Fresh?"
- Resume → decrypt → validate against Zod → restore full form state
- Start Fresh → clear all saved data

5.8 Anti-Screen-Capture for E-Signature

- CSS `backdrop-filter: blur(1px)` overlay on signature canvas
- Prevents visual capture of e-signature via screen recording

6. Testing & Quality Assurance

6.1 Test Coverage — 15 E2E Test Specs, 62+ Test Cases

P0 Tests (mandatory, 11 specs)	#	Test	What It Validates	--- ----- -----	1
Personal Loan Happy Path Full flow: salaried, submit, success modal with UUID 2 Home Loan Happy Path Co-applicant, property docs, passport 3 Business Loan Happy Path GST, ITR, business registration docs 4 Step 1 Validation Errors Empty submit, min/max amount, boundary test (₹5L threshold) 5 Step 2 Validation Errors Age <21, age >65, boundary (exactly 21), invalid email, mobile 6 Step 3 KYC Validation Invalid PAN format, wrong entity type, Aadhaar Verhoeff fail, missing consent 7 Step 4 PIN Code Lookup Valid PIN auto-fills, invalid PIN shows error, rent/previous fields 8 Step 5 Employment Switch Salaried → Self-Employed clears stale data, type cycling 9 File Upload & Compression JPG/PDF upload, compression feedback, missing doc error 10 Auto-Save & Resume Save → reload → resume modal → data restored 11 Cross-Step Dependency Loan type change affects subsequent steps, default Spouse					
P1 Tests (4 specs)	#	Test	What It Validates	--- ----- -----	12 Step 6 Conditional Visibility Home shows, small Personal hides, threshold testing 13 E-Signature Capture Draw → captured, clear → validation, redraw 14 Keyboard Navigation Complete form without

mouse (Tab, Shift+Tab, Enter, Space) | | 15 | Rapid Navigation Stress | Rapid clicks, cycling, data integrity |

6.2 Custom Test Infrastructure

- **14 custom Cypress commands:** `fillStep1` – `fillStep8` , `uploadDocument` , `drawSignature` , `setFormData` , `goToStep` , `triggerAutoSave` , `completeAllSteps`
- **3 full fixture datasets:** Personal (Rajesh Kumar), Home (Amit Sharma), Business (Priya Patel) with realistic PAN, Aadhaar, addresses
- **12 sample documents:** Aadhaar, PAN, salary slips, bank statements, IT returns, property docs, business registration, GST returns
- **Cypress bridge:** `window.__wizardGoToStep` for direct test navigation

6.3 Linting

- ESLint with `eslint-config-airbnb` , `eslint-plugin-react-hooks` , `eslint-plugin-jsx-a11y` , `eslint-plugin-import`
- Zero errors in production build
- `no-console` rule (allows only `warn` / `error`)

7. Security & Compliance

7.1 Data Security

Measure	Implementation
Encryption	AES-256-GCM via Web Crypto API for all stored data
PII Masking	PAN shows last 5, Aadhaar shows last 4 (<code>.....</code>)
Anti-Screen-Capture	CSS blur overlay on e-signature canvas
Draft Expiry	Auto-delete after 72 hours
Data Cleansing	localStorage cleared on successful submission
Corruption Detection	Schema validation on resume rejects tampered data

7.2 Accessibility (WCAG 2.1 AA)

- All inputs have `<label>` via `htmlFor / id` pairing
- Error messages use `aria-live="polite"` + `role="alert"`
- Focus management on step transitions
- Full keyboard navigation
- Touch targets $\geq 44 \times 44$ px
- `autocomplete` attributes on all relevant fields
- Visible focus rings on all interactive elements
- Progress indicator with `aria-label`

7.3 Production Safeguards

- No `console.log` of PII in production
- ESLint enforces security best practices
- Input validation on both client side (Zod) and via schema factory

8. Skills & Technologies Gained

8.1 Technical Skills

Skill Area	Technologies/Concepts
Front-End Framework	React 19, JSX, Components, Hooks
Build Tools	Vite 8, PostCSS, Autoprefixer
Styling	Tailwind CSS 3, Responsive Design, Custom Brand Themes
Form Management	react-hook-form 7, Controlled/Uncontrolled Components
Validation	Zod 4, Dynamic Schema Generation, Cross-Step Validation
Testing	Cypress 15, E2E Testing, Custom Commands, Fixtures
Security	Web Crypto API, AES-256-GCM, Data Encryption
Accessibility	WCAG 2.1 AA, ARIA, Keyboard Navigation
Architecture	Compound Components, Custom Hooks, Container-Presenter
Version Control	Git, Conventional Commits, Feature Branches

8.2 Soft Skills

- Project planning and execution (3-sprint schedule)
 - Requirement analysis and implementation
 - Problem-solving and debugging
 - Documentation and knowledge transfer
-

9. Challenges & Solutions

9.1 Cross-Step Validation Complexity

Challenge: A change in Step 1 (loan type) can affect validation rules in Steps 5, 6, 7, and 8. Managing these dependencies cleanly without coupling steps together.

Solution: Implemented a three-layer architecture — Schema Layer (Zod `superRefine` with form state), State Layer (smart field clearing on change), UI Layer (reactive visibility computation). This separates concerns and keeps each step independent.

9.2 Dynamic Step Visibility

Challenge: Step 6 (Co-Applicant) can appear or disappear mid-flow based on loan amount and type. The step count changes, affecting progress calculation.

Solution: The Wizard computes visible steps via `useMemo` with `computeShowCoApplicant()`. Progress percentage and step indices adjust dynamically. Navigation guards prevent advancing to invisible steps.

9.3 Verhoeff Checksum Implementation

Challenge: Aadhaar validation requires the Verhoeff algorithm with specific `d`, `p`, and `inv` lookup tables. Getting the tables correct was critical.

Solution: Carefully implemented the three standard lookup tables and verified against known valid Aadhaar numbers. The checksum returns 0 for valid numbers.

9.4 Encrypted Auto-Save with Web Crypto API

Challenge: The Web Crypto API is async-only and requires specific key derivation. `localStorage` has 5MB limits with base64 overhead.

Solution: Used AES-256-GCM with a derived 32-byte key, 12-byte random IV, and base64 encoding. The total encrypted payload stays well under localStorage limits for the form data size.

9.5 Client-Side Image Compression

Challenge: Compressing images in the browser without blocking the UI thread, and providing meaningful size feedback to the user.

Solution: Used an async Canvas API pipeline with recursive quality reduction. Runs quality from 0.7 down to 0.3 in 0.1 steps until target size is met. Shows both original and compressed sizes.

10. Project Statistics

10.1 Code Metrics

Metric	Value
Total source files	36
Total test files	22
Lines of code (approx.)	~8,000
Git commits	65
E2E test specs	15
E2E test cases	62+
Custom test commands	14
Sample documents	12
PIN code entries	112

10.2 Commit Distribution

Category	Count	Examples
Features	20+	<code>feat(step1)</code> , <code>feat(auto-save)</code> , <code>feat(image-compression)</code>
Tests	15+	<code>test(cypress)</code> , <code>test(fixtures)</code>
Documentation	5+	<code>docs(readme)</code> , <code>docs(architecture)</code>
Infrastructure	10+	<code>chore(setup)</code> , <code>chore(config)</code>

10.3 Sprint Timeline

Sprint	Days	Focus	Deliverables
Sprint 1: Foundation	1–5	Setup, common components, Steps 1–4, PIN lookup	15 source files, 4 schemas, 3 hooks
Sprint 2: Complexity	6–10	Steps 5–8, cross-step logic, auto-save, file upload	15 more source files, 5 schemas, 2 hooks
Sprint 3: Quality	11–15	E2E tests (15+), accessibility audit, responsive polish	22 test files, 3 fixtures, 12 samples

11. Conclusion

The LendSwift Multi-Step Loan Application was successfully delivered with all 8 steps implemented, 14 cross-step dependencies handled, 15 E2E test journeys written, and full WCAG 2.1 AA accessibility compliance. The application demonstrates modern React patterns including compound components, custom hooks, dynamic schema generation, and encrypted client-side persistence.

Key Achievements

- **100% of planned features implemented**
- **15 E2E test suites** with 62+ individual test cases
- **AES-256-GCM encryption** for all persisted data
- **WCAG 2.1 AA** accessibility compliance
- **65 git commits** following conventional commit standards
- **Zero ESLint errors** in production build

Future Enhancements

- Server-side persistence with REST API
- GitHub Actions CI pipeline
- Multi-language support (i18n)
- Performance optimization (lazy loading step components)
- Integration with real verification APIs

Report prepared for Internship completion — LendSwift Multi-Step Loan Application